HSLU Hochschule Luzern

Multi-Stage Job Advertisement Analysis using BERT and Large Language Models

Final Project Documentation

W.MSCIDS_GEN03.F26 / Advanced Generative AI

Authors:

Maria Dafni Kokkoni
maria.kokkoni@stud.hslu.ch

Céline Schmied
celine.schmied@stud.hslu.ch

Shpetim Veseli
shpetim.veseli@stud.hslu.ch

Supervisor:

Dr. Marcel Blattner
marcel.blattner@hslu.ch

GitHub Repository:
https://github.com/MariaDaf/Skill-Extractor

08. June 2026

Table of Contents

Introduction

Job advertisements are a rich source of labour-market information, but that information is locked inside unstructured text. Recruiters, job seekers and labour-market analysts are all interested in the same questions about which skills are required, which qualifications are expected and which benefits are offered. The problem is that the text is messy. It is free-form, mixes different sections, varies a lot in length and follows no fixed schema. Reading thousands of ads by hand is not realistic, so the structure has to be extracted automatically.

The goal of this project, in one sentence, is to turn raw job advertisements into a clean, queryable list of skills automatically. We approach this with a two-stage NLP pipeline for German-language job advertisements that combines the two workhorses of modern NLP, a fine-tuned transformer (BERT) and a general-purpose Large Language Model (Gemini). In the first stage, a fine-tuned BERT model classifies each token of an advertisement into a semantic zone, for example "Fähigkeiten und Inhalte", "Anstellung" or "Benefits". In the second stage, the text that was identified as the skills zone is passed to the LLM (Google Gemini), which extracts concrete professional skills as short phrases of two to five words.

The project is divided into five tasks and was built by three team members who worked one after another. Person 1 prepares and annotates the data, Person 2 trains and evaluates the BERT zone classifier, and Person 3 implements the web crawler and the LLM based skill extraction pipeline. The rest of this report follows this division into three parts, after the shared problem statement below.

Problem Statement

Starting from an unstructured German-language job advertisement, the project deals with two connected sub-problems. The first is to automatically find the section that describes the required skills and content ("Fähigkeiten und Inhalte"). The second is to extract a clean and concise list of professional skills from that section, written as phrases of two to five words. In short, the aim is to convert messy, free-form job ads into structured, queryable skill data.

We split the task into two stages on purpose, because a single model does not handle both sub-problems well at the same time. Finding the relevant section is a token-level sequence-labelling problem, which fits a fine-tuned BERT classifier. Extracting well-formed skill phrases is a generative language task, which a Large Language Model handles better. This also lets us see where each of the two NLP approaches shines. One important challenge of this design is that mistakes in the first stage, especially imprecise zone boundaries, carry over into the second stage and limit the quality of the extracted skills. Part 3 looks at this dependency in detail.

Part 1 — Data Preparation & Preprocessing

1 Objective

The objective of Part 1 is to transform the raw annotated dataset into a format that the BERT zone classifier (Part 2) can train on directly. This involves understanding the structure of the annotations, implementing a preprocessing pipeline that converts character-level label spans into token-level labels, and producing the datasets and label mapping that all subsequent parts of the project depend on.

2 Dataset Description

2.1 Source and Structure

The dataset is provided in annotated.json, created using the Label Studio annotation tool. It contains German-language job advertisements collected from online sources, each paired with manually assigned zone labels. Two fields are relevant for preprocessing:

- data['content_clean'] — the full raw text of the job advertisement as a single string
- annotations[0]['result'] — a list of labeled spans, each specifying a start and end character offset within content_clean and a zone label

Annotations are character-level: each span records the exact position in the raw text string where a zone begins and ends. The preprocessing pipeline must translate these character positions into the token-level indices that BERT requires.

2.2 Size and Quality

The raw dataset contains 2,699 entries. During exploration, 11 entries were found to contain no annotation spans and were removed, leaving 2,688 usable job advertisements. The removed entries fell into three categories:

- French-language ads (5 entries): these had no annotations, so all their tokens would have been incorrectly assigned the label O during training. Note that the language itself was not the problem — bert-base-multilingual-cased supports French — but training on completely unannotated text teaches the model the wrong patterns.
- HTML/URL-encoded text (2 entries): unreadable content resulting from incomplete crawler cleaning, with raw HTML entities such as %26 and %3CDIV in place of readable text.
- Corrupted entries (4 entries): system error pages and crawler artifacts with no meaningful job advertisement content.

Metric	Value
Raw entries	2,699
Removed (unannotated)	11
Clean dataset	2,688
Total labeled spans	19,666
Shortest / longest job ad (after cleaning)	101 / 19,598 chars
Average length	1,591 characters
Entries exceeding 512 tokens (estimated)	~687 (25.6%)

2.3 Zone Labels

The dataset contains 10 distinct zone labels. The project specification lists 8 — two additional categories, *Firmenkundenbeschreibung* and *Arbeitsumfeld*, were discovered during data exploration and included in the label mapping. All 10 are retained because excluding them would cause those spans to be mislabeled as O during training.

Label	Count	Share
Fähigkeiten und Inhalte	5,208	26.5%
Anstellung	4,001	20.3%
Bewerbungsprozess	2,651	13.5%
Abschlüsse	2,256	11.5%
Erfahrung	1,918	9.8%
Benefits	1,391	7.1%
Firmenbeschreibung	1,303	6.6%
Firmenkundenbeschreibung	769	3.9%
Challenges	108	0.5%
Arbeitsumfeld	61	0.3%
O (Outside — assigned in code, not by annotators)	—	—

The label distribution reveals a significant class imbalance. *Challenges* and *Arbeitsumfeld* together account for less than 1% of all annotated spans. Without countermeasures the model would almost never learn these zones. This is addressed during training using class-weighted CrossEntropyLoss (Part 2).

2.4 Annotation Quality

The character offsets in each annotation (value['start'] and value['end']) were verified to reliably locate spans within content_clean. A minor inconsistency was found in the value['text'] field, which stores escaped newlines as the two-character sequence \\n rather than as real newline characters. This causes string mismatches when value['text'] is compared directly to content_clean. Since the pipeline uses only the character offsets and never value['text'], this inconsistency has no effect on label alignment.

One additional structural observation was made during exploration: annotations within a single job ad are not always sorted by character position, and the same zone label can appear multiple times in non-contiguous positions. Both cases are handled correctly by the preprocessing logic, which processes each span independently and iterates through all tokens for every annotation.

3 Methodology

3.1 Script

The preprocessing script is based on the professor-provided preprocessing.py. Three targeted improvements were made to fix bugs and align the script with standard PyTorch practice. These are described in detail in Section 4. The script is run from the scripts/ subfolder and uses relative paths to read from data/ and write to data/ and model/.

3.2 Pipeline Steps

The pipeline transforms the raw JSON dataset into PyTorch TensorDatasets through six sequential steps:

#	Step	Description
1	Load & filter	Read annotated.json via pandas; remove 11 unannotated entries
2	Tokenize	BertTokenizerFast splits text into sub-word tokens; offset_mapping links each token to its character position
3	Label alignment	find_token_span() maps character-level annotation spans to token indices; unmatched tokens receive label O
4	Sliding window	Job ads longer than 510 tokens are split into overlapping chunks (stride=256) to stay within BERT's 512-token limit
5	Label encoding	String labels converted to integer IDs; label2id and id2label built dynamically and saved to id2label.json
6	Dataset creation	80/20 train/test split; sequences padded and converted to PyTorch TensorDatasets; saved as .pt files

3.3 Key Design Decisions

Tokenizer choice
BertTokenizerFast is a hard requirement, not an optional preference. Only the Fast variant of the tokenizer provides offset_mapping, which is the key that links each token back to its character position in the original text. Without offset_mapping, there is no way to know which character range a given token covers, and label alignment becomes impossible. The base BertTokenizer does not provide this feature. The multilingual-cased variant was selected because the job advertisements are in German: the model handles umlauts (ä, ö, ü, ß) and preserves capitalisation.

Label alignment and the fallback mechanism
For each token, its character range is checked against all annotation spans. If the token falls entirely within a span, it receives that span's label. If no span matches, it receives O. In practice, annotation boundaries do not always fall exactly on token boundaries — they sometimes land on whitespace between tokens, or between two sub-word pieces of the same word. In these edge cases, a fallback mechanism finds the nearest token rather than silently dropping the annotation. This prevents annotation loss at zone boundaries, which would be especially harmful for short zones such as *Challenges* and *Arbeitsumfeld*.

Sliding window with overlap
Approximately 25.6% of job advertisements exceed BERT's 512-token limit. Simply truncating these would discard all annotations in the later sections of the ad. Instead, a sliding window splits long documents into overlapping chunks of up to 510 tokens, advancing by 256 tokens at each step. The 254-token overlap ensures that tokens near a chunk boundary appear in at least two chunks, each time with different surrounding context. Each chunk is treated as an independent training example by Part 2.

Label padding with -100
When sequences within a batch are padded to a uniform length, the padding positions in the label tensor are assigned the value -100. PyTorch's CrossEntropyLoss has a built-in ignore_index parameter that, when set to -100, completely excludes those positions from the loss calculation. The original script used label2id['O'] (which equals 0) as the padding value, but this is ambiguous: the model cannot distinguish a genuine O token from a padding token, and would attempt to learn from padding positions. The project's own data_preparation_doc explicitly acknowledged this issue, noting that -100 would align more directly with common CrossEntropyLoss practice.

Robust annotation filter
The original filter (lambda x: len(x[0]['result']) > 0) would raise a TypeError if the annotations field was not a list, an IndexError if the list was empty, or a KeyError if the result key was absent. The improved filter adds four sequential checks before accessing x[0]: it verifies that the field is a list, that the list is non-empty, that the first element is a dictionary, and that the result key exists using .get() with a safe default. This made the filter robust to all malformed entries present in the dataset.

4 Experimental Setup and Results

4.1 Environment

Component	Details
Python version	3.12.5
Key libraries	transformers, torch, pandas, scikit-learn
Tokenizer	bert-base-multilingual-cased (BertTokenizerFast)
Hardware	MacOS, CPU
Script	scripts/preprocessing.py
Run command	python preprocessing.py (from scripts/ folder)

4.2 Output Statistics

The script was run on the full clean dataset of 2,688 job advertisements. The sliding window expanded this into 3,741 training examples (chunks), since job ads longer than 510 tokens produce more than one chunk. The 80/20 split was applied to these chunks.

Metric	Value
Job ads processed	2,688
Total chunks produced (after sliding window)	3,741
Training chunks (80%)	2,992
Test chunks (20%)	749
Tensor shape	(n_chunks, 510)
Number of unique labels	11 (10 zones + O)
Label padding value	-100
train_dataset.pt file size	35 MB
test_dataset.pt file size	8.7 MB

5 Challenges Encountered

Several challenges arose during data preparation and are worth documenting.

Tokenizer API incompatibility. The professor-provided script called tokenizer.encode_plus(), which is not available on the Fast tokenizer backend. This caused every job advertisement to fail silently inside a try/except block, producing an empty dataset with no error visible to the user. The bug was identified by inspecting the preprocessing.log file, which showed the same AttributeError repeated for every row. The fix was a one-line change to the direct tokenizer() call, which is the current Hugging Face API and functionally equivalent.

Undocumented labels in the data. A dataset-wide scan revealed 10 distinct zone labels rather than the 8 described in the project specification. The two additional labels, Firmenkundenbeschreibung and Arbeitsumfeld, were not mentioned in the brief. Had they gone undetected, those spans would have been mislabeled as O during training, silently corrupting the label alignment for roughly 4.2% of all annotated spans. Both labels were added to the mapping.

Unannotated entries producing false O labels. Eleven job advertisements had no annotation spans. If kept in the dataset, every token in those ads would have been assigned label O, teaching the model that those texts contain no zones — which is incorrect (the ads were simply unannotated, not truly zone-free). All 11 were identified through a dataset-wide check and removed before preprocessing.

Ambiguous label padding. The original script used label2id['O'] (integer 0) as the padding value for label tensors. Since 0 is also a valid label for genuine O tokens, the Part 2 loss function would have been unable

to distinguish padding from real content, effectively training on padding positions. This was replaced with the PyTorch standard of -100, which CrossEntropyLoss excludes automatically via its ignore_index parameter.

Part 2 — BERT Training & Evaluation

1 Objective

The objective of Part 2 is to train a BERT-based token classification model that predicts the correct zone label for each token in a job advertisement.

The model receives the preprocessed token-level datasets from Part 1 and learns to classify each token into one of the predefined job-ad zones, such as Fähigkeiten und Inhalte, Abschlüsse, Erfahrung, Benefits, Bewerbungsprozess, or O.

The output of this part is a trained BERT model that can identify relevant text zones in unseen job advertisements. The most important zone for the next project step is Fähigkeiten und Inhalte, because this zone is later used as the input for Gemini-based skill extraction.

1.1 Input Files from Part 1

The training script uses the following files produced during preprocessing:

File	Purpose
train_dataset.pt	Training data containing token IDs, labels, and attention masks
test_dataset.pt	Test/validation data used to evaluate the model
id2label.json	Mapping from numeric label IDs back to readable zone names

The datasets are loaded using torch.load(). The label mapping is loaded from id2label.json, then reversed to create label2id.

The number of labels is derived dynamically:
num_labels = len(id2label)
In this project, the final number of labels was: 11 labels

2 Model Setup

The model used for training is:
bert-base-multilingual-cased

It was selected because the job advertisements are mainly German-language texts, and the multilingual cased BERT model can handle German characters, umlauts, and capitalization.

The model was initialized as:

```
BertForTokenClassification.from_pretrained(
    MODEL_NAME,
    num_labels=num_labels,
    id2label=id2label,
    label2id=label2id
)
```

This adds a token-classification layer on top of BERT. The pretrained BERT layers provide language understanding, while the new classification layer learns the project-specific zone labels.

During initialization, warnings appeared that classifier.weight and classifier.bias were newly initialized. This is expected because the final classifier layer is specific to our 11 zone labels and must be trained from scratch.

2.1 Training Environment

Training was executed locally using an NVIDIA GPU.

The environment check showed:

PyTorch version: 2.5.1+cu121
CUDA available: True
GPU: NVIDIA GeForce RTX 4060

The script automatically selects GPU if available:
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

The final training run used:
Using device: cuda

This was important because BERT training on CPU was too slow.

2.2 Training Configuration

The final training configuration was:

Parameter	Value
Model	bert-base-multilingual-cased
Epochs	3
Batch size	8
Gradient accumulation steps	2
Effective batch size	16
Learning rate	2e-5
Optimizer	AdamW
Scheduler	Linear scheduler with warmup
Warmup ratio	10%
Validation interval	Every 50 training steps
Loss function	Weighted CrossEntropyLoss
Padding ignore index	-100

Gradient accumulation was used to simulate a larger batch size without requiring too much GPU memory.

3 Training Process

3.1 Class Imbalance Handling

The dataset is strongly imbalanced. Some labels occur very often, while rare labels such as Challenges and Arbeitsumfeld occur only a small number of times.

To address this, class weights were calculated from the training data. Labels with fewer examples received higher weights, so that the model would not ignore them during training.

The loss function was defined as:

```
torch.nn.CrossEntropyLoss(
    weight=class_weights,
    ignore_index=-100
)
```

The ignore_index=-100 setting is essential because Part 1 padded label sequences with -100. This prevents the model from learning from padding tokens.

The calculated class weights showed that rare classes received much larger weights. For example:
Challenges: high weight
Arbeitsumfeld: high weight

This confirms the strong class imbalance in the dataset.

3.2 Training Process

The training loop follows this process:

Load one batch
↓
Move input_ids, labels, and attention_mask to GPU
↓
Run BERT forward pass
↓
Calculate weighted loss
↓
Backpropagate loss
↓
Apply gradient accumulation
↓
Update model weights with AdamW
↓
Update learning rate scheduler
↓
Validate periodically
↓
Save best model if validation loss improves

The script saves two model checkpoints:

File	Meaning
best_student_model.pt	Model checkpoint with the best validation loss
final_student_model.pt	Final model after training finishes

The best model is the most important output for the next project phase.

3.3 TensorBoard Monitoring

TensorBoard was used to monitor the training process.

The following metrics were logged:

Loss/train_step
Loss/validation_step
Loss/validation_epoch
Accuracy/validation_step
Accuracy/validation_epoch
Learning_rate

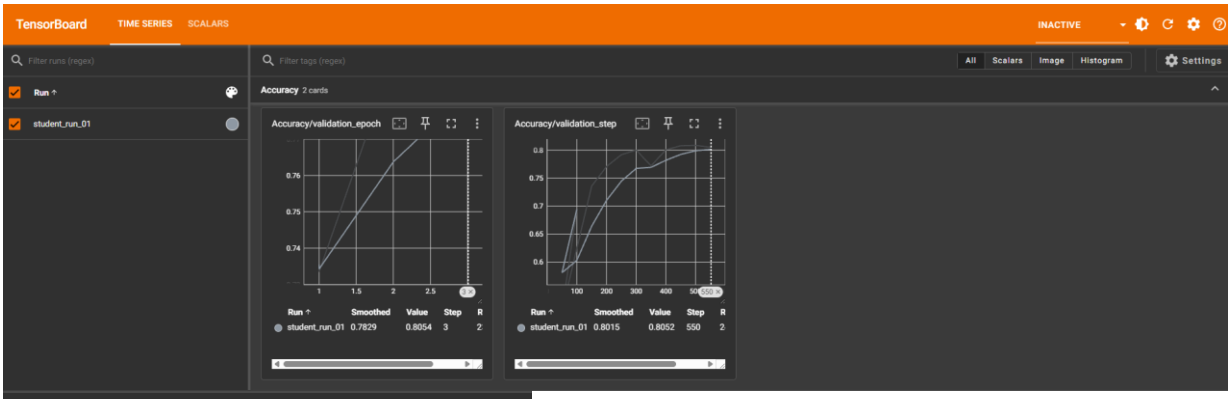
The TensorBoard curves showed that:

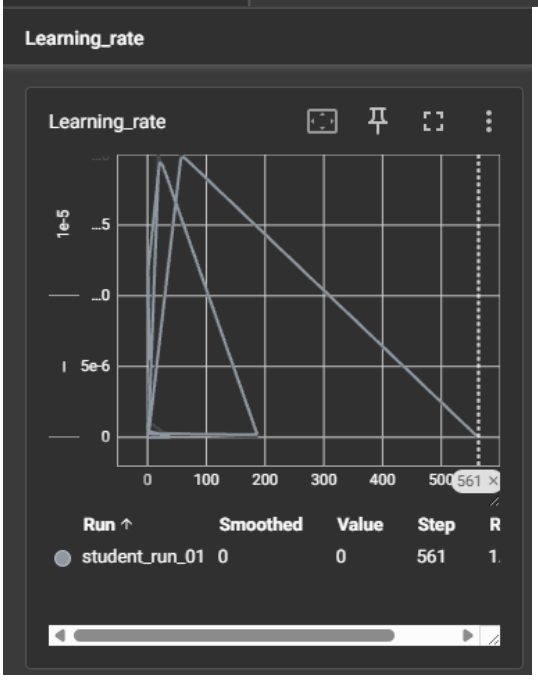
Training loss decreased over time.
Validation loss also decreased.
Validation accuracy increased.

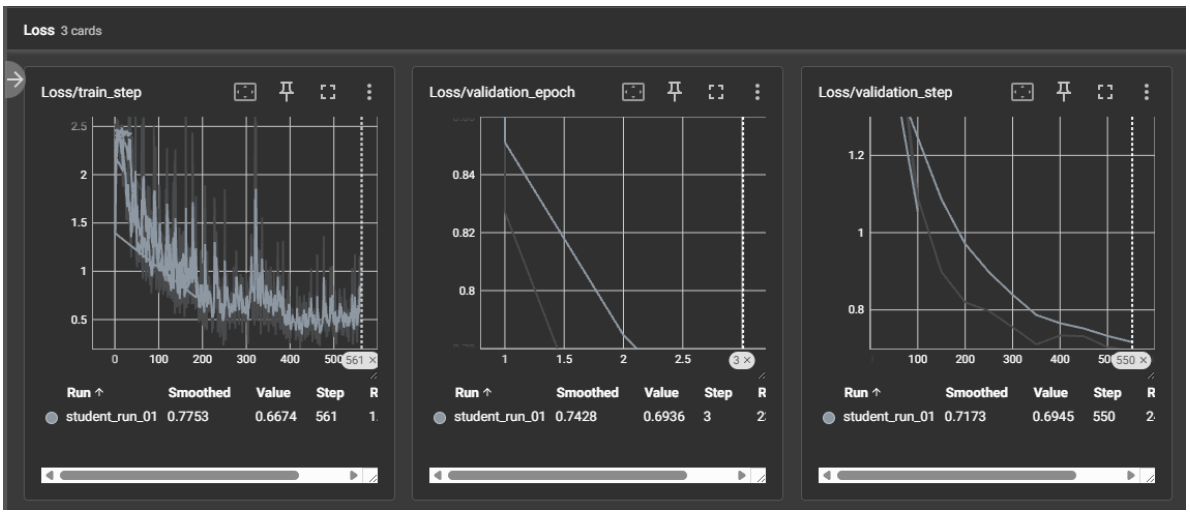
This indicates that the model learned useful patterns from the training data and that the training process was stable.

The validation accuracy reached approximately:
Validation accuracy: around 0.80

The validation loss decreased to approximately:
Validation loss: around 0.69







4 Evaluation and Results

4.1 Final Evaluation

After training, the best model checkpoint was loaded and evaluated on the test set.

In addition to validation accuracy, a stricter sequence-level evaluation was performed using seqeval.

The final result was:

Metric	Value
Precision	0.1746
Recall	0.3190
F1-score	0.2257

The detailed classification report showed that some labels were easier to predict than others. Frequent labels had better support, while rare labels such as Arbeitsumfeld and Challenges remained difficult.

4.2 Interpretation of Results

The model successfully trained and evaluated end-to-end. The decreasing training and validation loss show that the model learned from the data.

However, the final seqeval F1-score is still relatively low. This means that exact zone-span prediction remains challenging.

The difference between validation accuracy and F1-score is important:
Validation accuracy measures token-level correctness.
Seqeval F1-score measures stricter zone-span quality.

A model can achieve reasonable token-level accuracy while still struggling to predict exact zone boundaries. This is especially true for long job advertisements, overlapping zones, rare labels, and semantically similar categories.

Therefore, the result should be interpreted as:
The BERT training pipeline works successfully, but the model quality still leaves room for improvement.

4.3 Conclusion

Part 2 successfully implemented the BERT training and evaluation pipeline for job-ad zone prediction.

The model was trained using GPU acceleration, class-weighted loss, a learning-rate scheduler, and TensorBoard monitoring. The best model checkpoint was saved based on validation loss, and final evaluation was performed using precision, recall, and F1-score.

Although the final F1-score shows that zone prediction remains challenging, the project goal for this stage was achieved: a working BERT-based zone classifier was trained, evaluated, and prepared for handover to the Gemini skill extraction step.

4.4 Limitations

Several limitations were observed:

The dataset is imbalanced. Rare labels such as Challenges and Arbeitsumfeld have very few examples. Some zone labels are semantically close, for example Firmenbeschreibung, Firmenkundenbeschreibung, and Arbeitsumfeld. Token-level accuracy can look stronger than span-level F1 because exact zone boundaries are harder to predict. The model was trained as a baseline and not extensively optimized. Further experiments with capped class weights, different learning rates, or more detailed error analysis could improve performance.

Part 3 — Web Crawling and Gemini Skill Extraction

1 Objective

Part 3 covers the two final tasks of the project. Task 4 collects a fresh, unlabeled set of real-world job advertisements through web crawling, and Task 5 builds the skill-extraction pipeline that turns an advertisement into a clean list of professional skills. The pipeline combines the trained BERT zone classifier from Part 2 with the Gemini Large Language Model: BERT isolates the section labelled “Fähigkeiten und Inhalte”, and only that text is sent to Gemini, which returns skills as short phrases of two to five words. Sending only the isolated section keeps the input focused and reduces the amount of text passed to the LLM.

2 Methodology

2.1 Web Crawling

Platform Selection

Several data sources were evaluated before a final choice was made. The selection process itself became an important part of the task, because it surfaced both legal and technical constraints that the project specification explicitly asks to respect (robots.txt, terms of service, polite crawling).

SBB career portal: The detail pages are hosted on prospective.ch, whose robots.txt explicitly permits crawling (“User-agent: * / Disallow:”). The portal offers many German-language ads, which matches the multilingual BERT model used in Tasks 1–3.

Choosing a source whose operator clearly allows automated access keeps the project on the ethically and legally safe side, which is exactly the behaviour the specification's "Ethical Considerations" section requires.

Crawler Implementation

The crawler (task4_crawler.py) uses the libraries named in the specification, requests for HTTP access and BeautifulSoup4 for HTML parsing, and follows a two-stage workflow.

- Stage 1 – Listing:The public job-search JSON feed of the SBB portal is fetched, returning all open positions with their detail-page links and a language flag.
- Stage 2 – Detail extraction:Each German-language detail page is downloaded and parsed. Instead of scraping fragile HTML structures, the crawler reads the embedded schema.org JobPosting block (application/ld+json), which cleanly separates the fields title, responsibilities and qualifications.

Data Cleaning and Ethical Measures

The description fields are delivered as HTML, so a cleaning step (clean_text) removes the markup, resolves HTML entities and normalises whitespace while keeping the paragraph structure. The following ethical measures are implemented.

- robots.txt is checked programmatically before any request is sent.
- A delay of 1.5 seconds is inserted between requests to avoid overloading the server.
- An honest, descriptive User-Agent identifies the crawler as an educational student project.
- Only publicly available job-posting text is collected, with no personal data gathered.

2.2 Skill Extraction Pipeline

BERT Zone Inference

The trained model is loaded from best_student_model.pt. The checkpoint is a dictionary that also contains the id2label mapping saved during training, so the predicted integer label IDs are converted back to label strings directly from the checkpoint. This is functionally equivalent to loading the separate id2label.json file from the handover notes, but avoids the risk of model and mapping diverging. For each ad, the model predicts one zone label per token, and all tokens predicted as “Fähigkeiten und Inhalte” are collected. Because this

zone can appear several times within one ad, the reconstruction logic collects every separate occurrence as its own segment and joins them. WordPiece sub-tokens (for example “##ung”) are merged back into whole words.

Prompt Engineering

A single, carefully engineered prompt is sent to Gemini for each ad. Following the specification, it combines the following elements.

- Role setting: The model acts as an expert HR analyst specialising in skill identification.
- Input specification: The text is declared to be an excerpt from the skills section of a job ad.
- Task definition: Identify and extract all relevant professional skills.
- Output format: Return the result strictly as a JSON list of strings.
- Skill constraint: Each skill must be a phrase of two to five words.
- Few-shot examples: Two input/output examples (one English, one German) guide the format.
- Negative constraint: Single words must not be extracted.

Response Parsing and Error Handling

Gemini's response is parsed into a Python list. The parser tolerates both a clean JSON list and a fenced code block, and falls back to line-by-line parsing if needed. It then enforces the constraints in code, keeping only phrases of two to five words and removing duplicates case-insensitively. API calls are wrapped in error handling with retries and exponential back-off (waiting 2s, 4s, 8s), so a transient network problem or a 429 rate-limit response does not lose the ad; only after three failed attempts is that single ad skipped. Ads for which BERT detects no skills zone at all are recorded with an empty skill list instead of causing an error.

2.3 Key Design Decisions and Justifications

The most important design choices in Part 3, together with the reasoning behind them, are summarised below.

- Crawling source: The SBB portal was chosen because its host explicitly allows crawling and offers many German-language ads matching the BERT model. Sources that disallow access were rejected rather than circumvented.
- JSON-LD instead of HTML: Reading the standardised schema.org block is far more robust than parsing HTML tables, because the HTML templates differ between employers while the JSON-LD structure does not.
- Libraries (requests, BeautifulSoup4): Used as required by the specification; requests handle the HTTP access and JSON feed, BeautifulSoup locates the JSON-LD block in each page.
- Targeted LLM input: Only the BERT-isolated skills zone is sent to Gemini, not the whole ad. This reduces token usage and focuses the model on the relevant section.
- Prompt design strategy: Role setting, a strict JSON output format, the two-to-five word constraint, two few-shot examples and a negative constraint together push the model towards clean, consistent skill phrases.
- Secure API key handling: The Gemini key is read from an environment variable and never hard-coded, as required by the specification.

2.4 Experimental Setup

The skill-extraction pipeline was run locally in a Python 3.12 environment. The relevant software components for Part 3 are listed below.

Component	Value
Programming language	Python 3.12
Crawling libraries	requests, BeautifulSoup4
Inference libraries	PyTorch, Hugging Face Transformers
BERT model	bert-base-multilingual-cased (loaded from best_student_model.pt)
LLM	Google Gemini (gemini-2.5-flash-lite)
LLM SDK	google-generativeai
API key handling	Environment variable (GEMINI_API_KEY), not hard-coded
Request delay / retries	4 s between calls, up to 3 retries with exponential back-off

The BERT inference itself does not require training in this part, since the model from Part 2 is reused; it runs on CPU, which is sufficient for inference on single advertisements. The training hyperparameters and hardware for the BERT model itself are documented in Part 2.

3 Results

3.1 Crawling Output

The crawled dataset consists of 30 German-language job advertisements collected from the SBB career portal. In contrast to the annotated training dataset, it has no ground-truth zone labels and is used purely as pipeline input, not for training. Each entry contains the following fields:

Field	Description
job_id	Unique identifier (crawl_001 … crawl_030)
titel	Job title
arbeitgeber	Employer (SBB AG)
ort	Location
land	Country (Schweiz)
link	Source URL
content_clean	Cleaned advertisement text

The collection process itself can be summarised in three stages, from the full search feed down to the cleaned ads that were saved:

Stage	Count
Positions returned by the search feed	149
After German-language filter	122
Cleaned ads collected and saved	30

The advertisements vary in length and cover a range of professional fields (for example maintenance, IT, controlling and engineering roles), which makes the dataset a realistic test of the pipeline on diverse, contemporary input. The cleaned text length per ad ranged roughly from 900 to 2,300 characters.

3.2 Skill Extraction Output

The extracted skills are saved as extracted_skills.json (zone text plus skill list per ad) and extracted_skills.csv (one job_id/skill pair per row), each linked back to its source ad. Across the 20 successfully processed ads, 298 skill phrases were extracted in total.

3.3 Practical Considerations: API Model and Quota

Two real-world API constraints were encountered and resolved during development, which the specification's "Practical Considerations" section explicitly anticipates.

- Model deprecation: The initially configured model (gemini-2.0-flash) returned a zero free-tier quota because it had been deprecated, and was replaced by a current free-tier model (gemini-2.5-flash-lite).
- Daily quota: The free tier allows only 20 requests per project per day. After this limit was reached, the remaining ads returned no skills, so 20 of the 30 ads were processed successfully. This is sufficient to demonstrate the pipeline, and the limit is a quota constraint, not an implementation fault.

4 Quality Evaluation of the Extracted Skills

The pipeline was run on 20 crawled advertisements and produced 298 skill phrases in total. Because there is no ground-truth skill list for the crawled ads, the quality of this output was assessed by manual review of a representative sample, as suggested in the specification. The review separates the quality of the BERT-isolated zone (the input to Gemini) from the quality of Gemini's extracted phrases (the output). The following examples illustrate both a successful extraction and cases where the upstream zone model degraded the input.

4.1 Successful Case (crawl_001)

For a maintenance technician position ("Mechaniker:in"), BERT isolated a clean, coherent skills zone, and Gemini returned well-formed phrases that map directly onto the advertisement, such as "gesetzlichen Instandhaltungsvorgaben", "elektrischen und thermischen Schienenfahrzeugen", "Umbauarbeiten am Rollmaterial" and "PC-Anwenderkenntnisse". Here the full chain worked as intended: the zone was reconstructed cleanly and every extracted phrase reflects an actual requirement from the original ad.

4.2 Degraded Input (crawl_002, crawl_003)

In other ads the BERT-isolated zone was fragmented and partly unreadable. In crawl_002 the reconstructed text contained broken fragments such as "gle stehst du", "d : innen" and "be ältigen", where the model picked only scattered pieces of words. In crawl_003 the SAP-related skills appeared with unknown-token gaps, for example "fundierte Kenntnisse in SAP [UNK] … in R/3 als auch in S/4HANA [UNK]". Despite this noisy input, Gemini still produced plausible phrases, for instance "Reiseverhalten ermitteln" and "digitale Arbeitsmittel beherrschen" for crawl_002, which shows some robustness in the LLM stage. The fragmentation does, however, limit how complete and reliable the final skill list can be.

4.3 Interpretation

The dominant factor limiting output quality is the zone-identification model, not the extraction prompt. In the Task 3 evaluation the BERT model reached a span-level seqeval F1-score of only 0.23 (precision 0.17, recall 0.32), while its token-level validation accuracy was considerably higher at around 0.80. This gap explains the fragmented zones: the model often labels individual tokens correctly, but predicts the exact boundaries of a span poorly, so it picks scattered tokens rather than a clean block. Gemini can then only work with what it receives. The workflow is therefore implemented correctly, but its end result is bounded by the upstream model. The Gemini stage itself performed well, applying the two-to-five word constraint consistently and tolerating noisy input better than expected.

4.4 Suggested Improvements

Three concrete improvements follow from this analysis.

- Improve the BERT zone model: This would have the largest effect on final skill quality, since it is the main bottleneck.
- Add a post-processing filter: Obviously broken fragments, for example segments containing [UNK], could be discarded before they reach the LLM.

- Validate against a skill taxonomy: Extracted skills could be checked against a known skill taxonomy to filter overly generic phrases.

5 Challenges Encountered

Several practical challenges arose during Part 3 and were addressed as follows:

- Blocked data sources: The first two candidate sources disallowed automated access in their robots.txt. This was resolved by respecting the restriction and switching to the SBB portal, whose robots.txt explicitly permits crawling.
- Inconsistent HTML templates: Different employers on the same platform use different HTML structures, which made direct HTML scraping unreliable. This was mitigated by parsing the standardised schema.org JSON-LD block instead, which is identical across templates.
- LLM model deprecation: The initially chosen model (gemini-2.0-flash) had been deprecated and returned a zero quota. It was replaced by a current free-tier model (gemini-2.5-flash-lite).
- API rate limits: The free tier allows only 20 requests per day. This limited the run to 20 of the 30 ads. The retry-and-back-off logic prevented crashes, and the limitation was documented rather than worked around, since 20 ads are sufficient to demonstrate the pipeline.
- Fragmented BERT zones: The weak zone-classification model (F1 0.23) produced fragmented input for some ads, including [UNK] tokens. This is analysed in detail in the Quality Evaluation and addressed there with concrete improvement suggestions.

Conclusion

This project implemented a two-stage NLP pipeline that turns unstructured German-language job advertisements into a structured list of professional skills. In the first stage, the raw dataset was explored and cleaned — two undocumented zone labels were discovered and incorporated, and 11 unannotated entries were removed to prevent label corruption. The cleaned data was then transformed into token-level training examples using a sliding window approach, producing 3,741 labelled chunks from 2,688 job advertisements (Part 1). A multilingual BERT model was fine-tuned to classify each token into a semantic zone (Part 2). In the second stage, the zone labelled "Fähigkeiten und Inhalte" was isolated and passed to the Gemini LLM, which extracted skills as short phrases (Part 3). A web crawler was additionally built to test the full pipeline on fresh, real-world advertisements from the SBB career portal.

The pipeline works end to end and demonstrates the intended division of labour between the two models. BERT locates the relevant section, and the LLM extracts well-formed skill phrases from it. The main limitation is the zone-classification model, which reached a span-level F1-score of only 0.23. This weakness propagates into the skill-extraction stage, where it produces fragmented input for some advertisements. The LLM stage itself proved robust, applying the two-to-five word constraint consistently even on noisy input.

Potential Future Work

Several directions could improve the pipeline in future work:

Part 1:

- **Rare label data collection.** Challenges and Arbeitsumfeld together represent less than 1% of all annotated spans. Even with class-weighted loss, extremely rare labels are hard to learn from so few examples. Future annotation effort could target job advertisements more likely to contain these sections.
- **Handling HTML and multilingual content.** Eleven entries were removed due to HTML-encoded content or non-German language. A more complete pipeline would decode HTML entities before annotation, potentially recovering those entries, and use language detection to flag non-German ads early.
- **Sliding window tuning.** The window size (510 tokens) and stride (256 tokens) were taken from the provided script without experimental tuning. Future work could evaluate different stride values and measure their effect on zone-boundary prediction quality, since a smaller stride increases context at chunk boundaries at the cost of more training examples.

Part 2:
Future work for Part 2 should focus on improving the BERT zone classifier, since Part 3 showed that the quality of Gemini's output depends strongly on the text extracted by BERT. When BERT identifies the Fähigkeiten und Inhalte zone clearly, Gemini can extract useful skills. However, if BERT produces fragmented or noisy text, the final skill list becomes less reliable. The main improvement would be better handling of class imbalance. Rare labels such as Challenges and Arbeitsumfeld are difficult for the model to learn, even with class weights. Future experiments could test capped class weights, oversampling rare labels, or merging very rare labels if they are not essential for the final goal. Another useful step would be manual error analysis. By comparing predicted labels with true labels token by token, we could better understand whether the model mainly struggles with wrong zone types, unclear boundaries, or fragmented predictions.Finally, a separate inference script should be created for easier handover. It should load the trained BERT model, take a new job ad as input, and return only the predicted Fähigkeiten und Inhalte text for Gemini. This would make the full pipeline easier to use without retraining the model.

Part 3:
Refine the LLM prompts: Few-shot examples drawn from the actual job-ad domain, or a more constrained output format, could make the extracted skills more consistent.

- Add post-processing of the zone text: A filter could remove fragments such as [UNK] tokens or residual HTML markup (for example a leftover "") before the text reaches the LLM.
- Evaluate skill quality more formally: Instead of manual review, the extracted skills could be validated against an established skill taxonomy such as ESCO to measure precision and coverage systematically.
- Use a paid or higher API quota: Processing all crawled ads (not only 20) would require a higher request limit, which would allow the pipeline to be evaluated at a larger scale.